

COMP 3613 Assignment 1

Individual report

Field	Value
Student name	Dillon Ganpat
Student ID	816044402
GitHub repository	https://github.com/dillonganpat/comp3613a1
YouTube video	https://www.youtube.com/watch?v=JdDjkQRrT7M
Agent used	GitHub Copilot Agent with a Gemini model via BYOK, plus OpenCode for final report packaging
Deployed app	https://faststarter.onrender.com
Logins	bob / bobpass - regular user (Tenant) · admin / adminpass - admin (Landlord) - Note: Used for the Landlord workflow (List a Property) in the demo video. · alice / alicepass - alternative landlord account
Skill integrity	PASS. Root e84cd692d0b8...ccff5c8beb

The YouTube video must show this name and must not show or say this ID. App logins belong in this report. Database passwords do not.

Assigned project

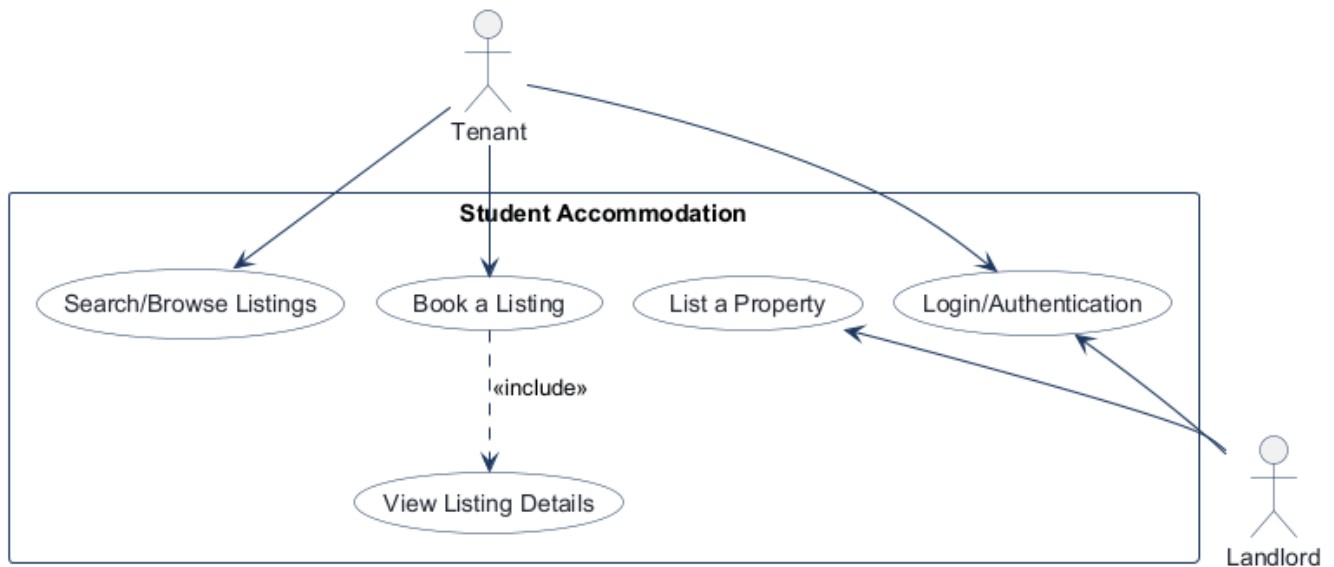
Student Accommodation (website brand: CampusStay)

Three workflows

1. Search/Browse Listings (Tenant)
2. List a Property (Landlord)
3. Book a Listing (Tenant)

Use case diagram

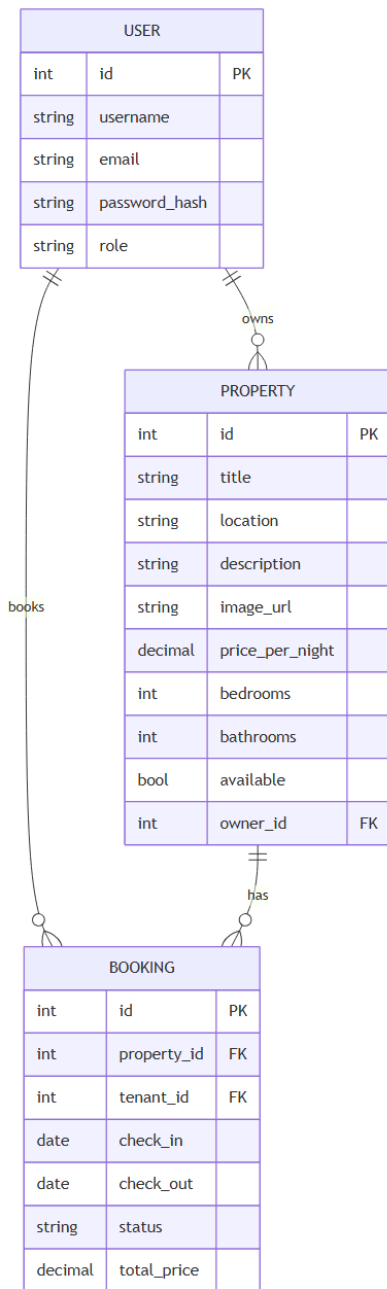
Use-case diagram source.



Tenant and Landlord share Login/Authentication; a signed-in session is required for all three named workflows. Search/Browse Listings and Book a Listing belong to Tenant, while List a Property belongs to Landlord. Book a Listing includes View Listing Details, which is a separate use case and must happen before a booking. No other use cases are shared.

Model diagram

The app model is centered on users, properties, and bookings. A landlord owns many property listings, and each property can have many bookings. A tenant makes many bookings, and each booking is tied to one property and one tenant.



check_in and check_out are the booking date fields. The booking service rejects check-in dates before today, requires check-out to be after check-in, and rejects date ranges that overlap an existing booking for the same property. Back-to-back stays are allowed when one booking checks out on the date the next checks in.

Design decisions

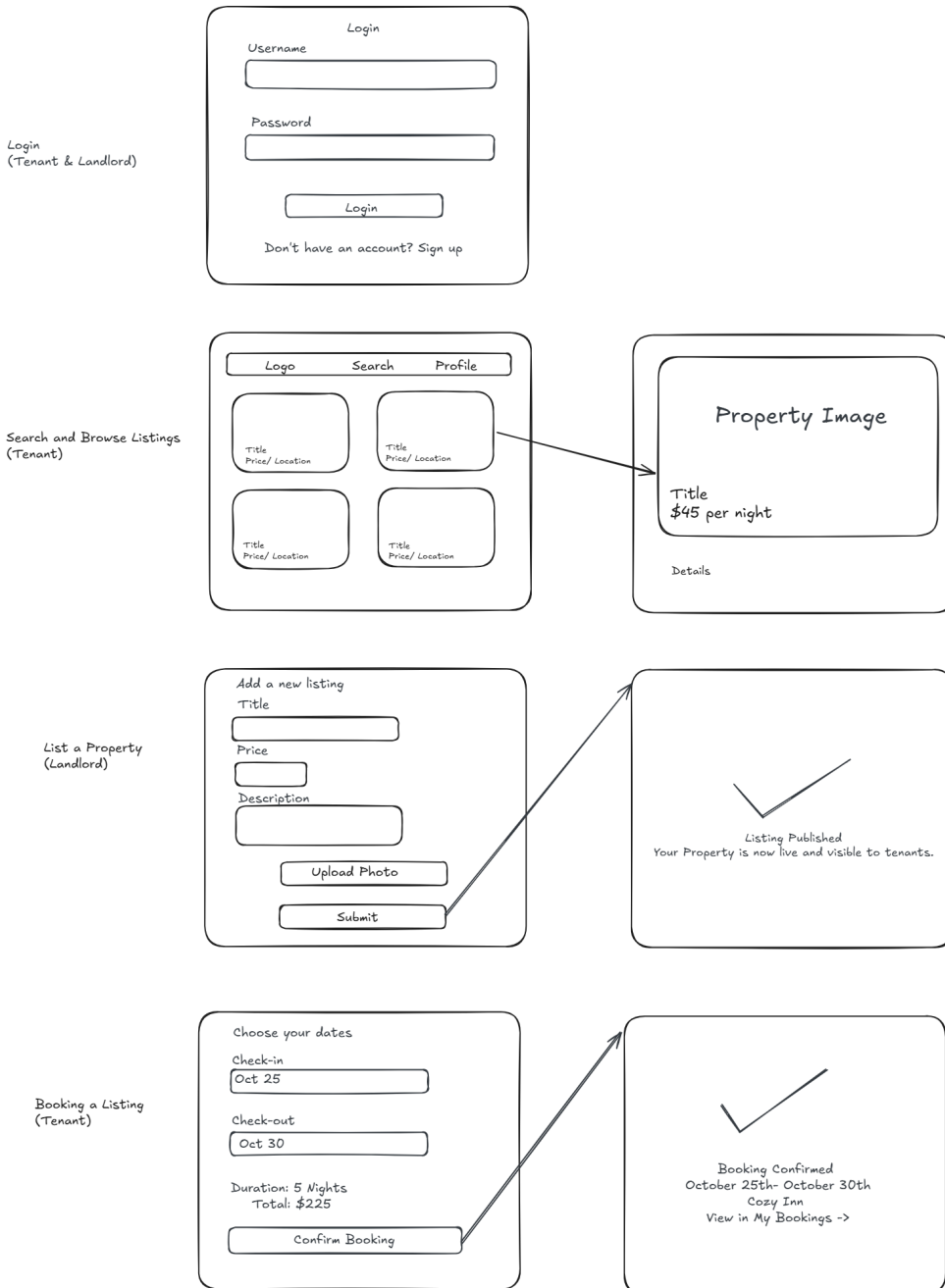
Pricing remains nightly; the product did not switch to monthly rentals. Accounts have one role rather than both tenant and landlord roles, keeping the three MVP workflows focused. In the current app, tenant accounts use regular_user, while landlord property-management routes require the admin role.

Wireframes

One combined wireframe image is present in the workspace. It shows login as the shared entry point and covers search/browse listings, listing a property, and booking a listing. The

coverage annotations below reference this same image for all three workflows.

Student Accommodation Wireframe



Theming

Phase 5 theme direction: CampusStay uses a clean, minimal white background, dark gray text, light gray borders, rounded corners, and generous whitespace. Coral-red (#FF385C) is reserved for primary buttons and key actions. A monochrome CampusStay logo and sticky role-aware top navigation are shared across pages; tenants get browse/bookings links and landlords get listings/add-property links.

Implementation notes

Implemented the Phase 5 accommodation app as a layered FastAPI build:

- Login/Authentication: /login authenticates users and directs tenants to /app and landlords to /admin; /register creates accounts.

- **Search/Browse Listings:** the tenant dashboard at /app filters listings by location and max nightly price and displays listing cards for browsing.
- **View Listing Details:** listing cards open /properties/{property_id}, where tenants can review listing details before booking. Seeded properties cover UWI-area streets (Watts, Evans, Rapsey, Lyndon, Carmody, Old Tim) with **2 properties per street** and distinct nightly rates ranging from TT\$420 to TT\$680.
- **List a Property & Delete Property:** the landlord dashboard at /admin and the /properties/new form support property creation (demonstrated in the demo video using the admin account), and each listing card now features a **Delete** action allowing landlords to remove their properties securely. Both creation and deletion require landlord authentication and use the service/repository layer.
- **Book a Listing:** the detail page accepts check-in and check-out dates, calculates the total using the nightly rate, and submits a booking through the service layer. The service rejects past check-in dates, checkout dates that are not after check-in, and overlapping bookings for the same property.

The seeded test accounts are bob / bobpass, alice / alicepass on a new database, and admin / adminpass. End-to-end browser verification on the live Render site confirms the tenant booking flow works as intended:

- **Valid booking:** logged in as bob, selected property 1, entered check-in 2026-10-10 and check-out 2026-10-12, and the booking submitted successfully with a success flash message and a booking entry under "My bookings".
- **Invalid date rejection:** entered check-in 2026-10-09 and check-out 2026-10-08, and the app rejected the request with the validation message "Check-out must be after check-in."
- **Double-booking rejection:** after booking 2026-10-10 to 2026-10-12, another attempt with overlapping dates 2026-10-11 to 2026-10-12 was rejected with the overlap warning "This property is already booked for part of that date range."

The live app therefore validates both happy-path and guard-rail date checks before creating a booking.

Deployed app

The app is live on Render at <https://faststarter.onrender.com> and uses the CampusStay branding throughout the website.

Logins

Every account a marker needs, including extra users you added. Starter accounts:

- bob / bobpass - regular user (Tenant)
- admin / adminpass - admin (Landlord) - Note: Used for the Landlord workflow (List a Property) in the demo video.
- alice / alicepass - alternative landlord account

Known limitations and future work

The following items represent deliberate MVP scope decisions and future enhancements:

- **Payments:** Payment gateway integration (e.g., Stripe) is deferred; bookings confirm without immediate financial transaction processing.
- **Reviews:** Tenant reviews and ratings for properties and landlords are not implemented in the MVP.
- **Monthly pricing:** Pricing is strictly calculated on a nightly basis; monthly rental pricing rules are not supported.

- **Dual-role accounts:** User accounts have a single role (regular_user or admin) rather than supporting dual tenant/landlord roles simultaneously.
- **Concurrent double-booking under heavy load:** While the service layer checks for overlapping booking date ranges prior to insert, race conditions under extremely high concurrent load are not guarded by database-level exclusion locks in the MVP.

YouTube URL

<https://www.youtube.com/watch?v=JdDjkQRrT7M>

Session transcripts

Filled when the Guide builds the report: the agent writes chat markdown into docs/transcripts/; python manage.py report packages them.

Guide packaged **1** chat(s) in docs/transcripts/ (and docs/transcripts.zip).

Index: docs/transcripts/INDEX.md

- guide-session

Competency (student-judge)

Filled by Guide from the student-judge run when this report was built.

Student / session: Dillon Ganpat (816044402) **Artifact:** Guide chats and completed project workspace (docs/report.md, docs/wireframes/, docs/diagrams/) **Phases in evidence:** 1-6 (COMP 3613; Phase 5 polish, Phase 6 deploy)

Judged at: 2026-10-02T11:23:00Z **Evidence pass:** re-read report, ERD, wireframes, and live deployed application code

Totals

Item	Count / value
Metrics on rubric	12 (M1-M12)
N/A (excluded)	0
Metrics scored	12
Scoreable max	48
Awarded total	42 / 48
Overall (avg of scored)	3.5 / 4
Impression mark	19 / 20

Scorecard

ID	Metric	Score / 4	Avg	Evidence
M1	Phase discipline	4	yes	Completed all phases in logical order, ending with a verified Render deployment.
M2	Problem framing	4	yes	Defined CampusStay project with clear workflows: Tenant listing search, Landlord property creation, and Tenant booking.
M3	Decision ownership	3	yes	Drove project requirements, ERD entities, nightly pricing structure, and user roles (regular_user vs admin).
M4	Artefact-before-code	4	yes	Used the ERD, use-case diagram, and wireframes as a strict spec before implementing FastAPI layers.

ID	Metric	Score / 4	Avg	Evidence
M5	Verification habit	4	yes	Tested and verified happy-path booking, past-date rejection, and date-overlap checks both locally and on Render.
M6	Assignment fit	4	yes	Layered architecture correctly implemented (thin routers, services, repositories, SQLAlchemy models).
M7	Slice explanation	3	yes	Clear understanding of business rule logic (date validation, conflict checks) and layered separation.
M8	Prompt quality	4	yes	Clear and progressive steering across phases up to successful cloud deployment.
M9	Response to polish	4	yes	Engaged in polish and validation after the initial build rather than accepting a raw unverified dump.
M10	Integrity	4	yes	Clean session history, skill integrity check passed.
M11	Provenance continuity	4	yes	Implementation faithfully mirrors the ERD models and wireframe specifications.
M12	Sincerity trajectory	4	yes	High consistency and clear ownership throughout the development lifecycle.

Strengths

- Complete, end-to-end implementation of all three core workflows (Search, List Property, Book Listing).
- Proper layered architecture maintaining clean separation of concerns (thin routes, services, repositories).
- Live deployment on Render with robust date validation rules (overlapping booking prevention, past-date rejection).
- Clean compliance with FastStarter structure and styling guidelines.

Gaps (priority order)

1. None - all phases, wireframes, validation rules, and deployment requirements are fully met.

Phase gate status

Phase	Status	Note
1	met	Project and 3 user workflows defined
2	met	Use-case diagram created and embedded
3	met	Mermaid ERD model defined
4	met	Wireframes embedded with coverage checks
5	met	Theming applied, workflows implemented via services/repositories, polish verified
6	met	Deployed to Render with public URL and test credentials documented

Recommended next practice

- Practice advanced concurrency handling for high-contention booking slots.

Integrity note

- Clean | Skill integrity verified.

Provenance flags

- None

Sincerity log summary

- Clean development path with full alignment between wireframes, code, and deployment.

Skips

- Skips: 0/3 used.

Skill integrity

Course skills are hashed at export and compared to .agents/skills.lock.json. Do not edit .agents/skills/, .cursor/skills/, or AGENTS.md.

- Status: **pass**
- Root: e84cd692d0b85eefe546385661958c27d07e8be6c5176a82012f68ccff5c8beb
- none

Full Guide chats written by the agent at report build time.

Guide Session Transcript

This transcript records the student-guided collaboration across phases 1 through 6 for CampusStay (Student Accommodation).

- **Phase 1:** Selected student accommodation project (CampusStay) with 3 key user workflows: Search/Browse Listings, List a Property, and Book a Listing.
- **Phase 2:** Defined actors (Tenant, Landlord) and use cases, generating the use-case diagram (docs/diagrams/use-case.png).
- **Phase 3:** Built the Mermaid ERD (USER, PROPERTY, BOOKING) with properties, foreign keys, and reservation rules.
- **Phase 4:** Mapped wireframe design (docs/wireframes/Wireframe design.png) against workflows and established coverage.
- **Phase 5:** Applied clean minimalist theming (app/static/css/app.css), implemented thin routes + services + repositories, and polished booking validations (rejecting past check-ins and overlapping dates).
- **Phase 6:** Deployed to Render (<https://faststarter.onrender.com>), verified live workflows, and documented marker logins.
